VerifierProbe: A Float32 Soundness Gap in α,β-CROWN's Bound Propagation

From hypothesis to prototype to a full probe of the CROWN core, with an introspection trace and an end-to-end check of the complete verifier

target: α,β-CROWN v0.8.0 (commit b58d83c), auto_LiRPA CROWN core · float32, CPU

Abstract. A formal verifier's certificate is only as sound as the arithmetic it runs on. Following Jia and Rinard [1], we asked whether α,β-CROWN [8], the repeated VNN-COMP winner, can certify a region as safe while the network it certifies mispredicts inside that region under real float32 execution. We formed a hypothesis from the paper and the verifier's source, tested it on a prototype, and then built `VerifierProbe`, a package with a verifier adapter, a grounded attacker, and an introspection prober. On a constructed network whose ReLUs are *proved* active over the whole box, so that CROWN's relaxation is exact in real arithmetic, float32 CROWN certifies a margin lower bound of +0.486 while a concrete input in the box has a float32 margin of −24 and flips the predicted class. Re-evaluating the same input in float64 gives +0.559, so the violation is purely floating point. An ordinary network shows no violation, and the complete α,β-CROWN pipeline still catches the constructed case through its attack stage, so the claim is a float32 discrepancy in CROWN's bound computation, not an end-to-end failure of the verifier.

Assignment requirement	How this work meets it	Where
Sound/complete; computation instrumented	Guarantees hold in real arithmetic; CROWN's backward bounds and intervals	§1
Grounded attack on a stated numerical gap	Backward vs forward float32 association; corner start, boundary search [1]	§1.2, §3.2
Verifier adapter; document the hooks	`certify()` over read-only auto_LiRPA entry points	§3.1
Documented introspection trace	Per-layer bounds, output coefficients, α slopes; `TRACE_SCHEMA`	§3.3
Honest negative result with near-miss	Natural control resists; full verifier falsifies the constructed case	§4, §5
Package, quickstart, tests, config, seed	Installable, pinned, dict config, seeded; 6 tests pass	§3, §4

Table 1. Assignment requirements mapped to this report.

1 Background and hypothesis

1.1 Sound, complete, and where the arithmetic lives

α,β-CROWN combines CROWN linear bound propagation [2], implemented in the auto_LiRPA library [3], with optimized relaxation slopes (α) [4] and branch-and-bound with per-neuron split constraints (β) [5]. It is **sound**: if it certifies that no input in the box X changes the prediction, none does. It is **complete**: given time, branch-and-bound resolves every ReLU property instead of answering "unknown". Both guarantees are proved for real-valued arithmetic. CROWN obtains a lower bound on the margin (true-class logit minus a competitor) by propagating linear coefficients backward from the output to the input, layer by layer, and then minimizing the resulting linear function over the box. That backward pass is the computation we instrument and attack. The source showed it runs in float32 unless a `double_fp` flag is set, and the repository ships a dedicated attack for VNN-COMP's SoundnessBench, so the developers treat numerical soundness as a live concern.

1.2 The hypothesis

Jia and Rinard [1] showed that verifiers reason in idealized arithmetic while deployed networks run in floating point, and that the gap can be exploited. Two of their findings shaped our plan. First, on naturally trained networks CROWN's relaxation is loose enough to absorb rounding error: it verified 29 of 32 original MNIST images but none of their near-boundary "quasi-safe" images. Second, they therefore attacked CROWN in a setting where the attacker chooses the network, keeping ReLUs always active and starting the search from a corner of the box. Later work extended the picture: rounding attacks on certified radii succeeded against β-CROWN on 2.6% of attacked images [6], and verifiers including β-CROWN were shown to be unsound against precision- and order-dependent backdoors [7]. We stated the hypothesis before writing code:

> **Hypothesis.** If the network is affine over X (every ReLU active), CROWN's relaxation is exact, so any disagreement with the network is floating point. If the margin is a small difference between two large logits, CROWN's backward pass (which subtracts the two large output rows first) and the forward pass (which computes both logits and subtracts last) round differently by about one ulp of the logits. An output bias placed inside that disagreement

should make float32 CROWN certify SAFE while a concrete input mispredicts. Predictions: the violation vanishes in float64, and an ordinary network shows none.

2 Prototype: testing the hypothesis in miniature

Before building anything, we ran the vendored auto_LiRPA directly on toy networks to see whether the gap exists and has the right sign. The first two designs failed, and each failure was informative (Table 2).

#	Construction	Outcome	Lesson
1	4-layer net, final layer scaled by $3{\cdot}10^4$	no bias produced a straddle	scaling the margin scales its error too; need two large logits that cancel
2	4-layer net with cancelling logits	gap ≈ +2.4, but minimum pre-activation −268	ReLUs switch off in the box, so the gap is not purely floating point
3	1 hidden layer (bias 10), cancelling logits, scale `big` swept	`big`=200: gap −0.019 (CROWN still sound); `big`≥1000: gap > 0; `big`=5000: straddle	the gap appears once one ulp of the logits ($\sim 2{\cdot}10^7$) exceeds the margin

Table 2. Prototype iterations. "Gap" is CROWN's float32 lower bound minus the float32 forward margin at the worst-case corner; it must be positive for a violation.

Design 3 confirmed the hypothesis. Float32 CROWN certified a margin lower bound of $+2.098{\cdot}10^{-4}$ (SAFE), while the forward pass at the corner gave −2.0 and the predicted class flipped. In float64 the bound ($+2.098407{\cdot}10^{-4}$) and the forward margin ($+2.098307{\cdot}10^{-4}$) agree to 10^{-10} and are both positive. The gap is therefore pure float32 rounding, and we proceeded to the full build.

3 Full implementation: the VerifierProbe package



Figure 1. The VerifierProbe pipeline. The first two stages build a calibrated instance; the last three are the required API.

3.1 Verifier adapter

`ABCrownVerifier.certify(model, X)` wraps the model in auto_LiRPA and computes a CROWN lower bound for every competitor class, using a specification matrix whose rows are $e_{\text{true}} - e_j$. The verdict is SAFE when the minimum bound is non-negative. We did not patch the verifier. The adapter reads only public entry points: `compute_bounds` for the bound itself, `return_A` for the output-to-input linear coefficients, and each node's stored `lower/upper` intervals. None of these change weights, relaxations, split decisions, or the verdict, so the certificate we read is the one the verifier produces unprobed.

3.2 Calibration and attack

Proof of activity. The prototype checked ReLU activity by sampling, which is not a proof. For an affine layer over an L_∞ box the minimum pre-activation of neuron i is exactly $b_i + W_i{\cdot}c - \varepsilon\|W_i\|_1$, so we compute it for every neuron and require the global minimum to be positive. **Boundary search.** Following the quasi-safe/quasi-adversarial search of [1], bisection locates two boundaries in output-bias space: where the float32 forward margin at the corner crosses zero, and where the float32 CROWN bound crosses zero. Each is localized to a bracket $\tau_1 - \tau_0 < 10^{-7}$. The width of the window between them, `fp_gap`, is a separate diagnostic, not $\tau_1 - \tau_0$. An early one-shot formula estimated that window from values at zero bias and overstated it (2.44 instead of 1.44) because the float32 forward margin moves in ulp-sized steps. The calibrated bias is the window midpoint. **Attack.** The search starts at the corner $x^* = c - \varepsilon{\cdot}\text{sign}(\nabla\text{margin})$, the exact minimizer of an affine margin, then runs a seeded hill-climb with a shrinking radius. Every accept decision uses a single-input forward pass, because batched matrix products round differently: during the build, a batched evaluation reported −14 for a point whose single-input prediction did not flip. A violation requires a SAFE certificate, a negative float32 margin, and a flipped class; the float64 margin of the same input is logged as a control.

Platform dependence. Which logit scale produces a straddle depends on the float32 math library. On Linux (PyTorch 2.14) `big` = 5000 straddles. On Windows (PyTorch 2.11) it does not (`fp_gap` −2.56), while 12500 and 17500 do and 7500, 10000, 15000, and 20000 do not. This matches the observation in [1] that each implementation has its own error pattern. The builder therefore tries 12500 first, falls back through a fixed list, and records every attempt.

3.3 Introspection trace

`Prober.dump(model, x)` returns a JSON-serializable trace: an ordered list of per-node intervals (input, hidden pre-activation, output), the output linear relaxation `lA`, `uA` (shape [1 × specs × 50]) and `lbias`, `ubias`, the certified bound, and α slopes when α-CROWN is used. Every field, with meaning and shape, is documented in `TRACE_SCHEMA`. The trace is computed for a box of radius ε centred on x, so for the adversarial input it reports 0.270 rather than the certificate's 0.486, which belongs to the original box.

4 Results

Quantity	Constructed network	Natural control
Global minimum pre-ReLU value	+9.6535 (all 64 ReLUs active)	—
Boundary search	boundaries −2.0000001 / −2.9725215 ($\tau_1-\tau_0 \le 8.9\cdot10^{-8}$); `fp_gap` +0.9725; bias −2.48626	—
Float32 CROWN lower bound (verdict)	**+0.48626 (SAFE)**	+1.546510 (SAFE)
Float32 margin at returned input	**−24.0** (logits 57,948,988 vs 57,949,012; class 1)	+1.546510
Float64 margin, same input	+0.5591	+1.546510
Soundness violation	**True**	False (near-miss reported)

Table 3. Main results. Seed 0, 50 inputs, 64 hidden units, ε = 0.05, hidden bias 10, `big` = 12500, CROWN in float32; Windows, Python 3.11.9, PyTorch 2.11, auto_LiRPA 0.8.0.

The constructed network behaves exactly as predicted. The hill-climb moved the float32 margin from −4 at the corner to −24. Because the network is affine on the box, the corner is the true minimum in real arithmetic, so any lower value elsewhere can only be rounding noise; the climb is a direct measurement of float32 error. A fixed seed reproduces the run within an environment; across installs the last digits differ (an earlier one found −20). In the natural control CROWN certifies 1.55 and the attack's best input has that same margin in both precisions: the relaxation's slack dwarfs any rounding error, as [1] reported for natural networks. All 6 tests pass: known-safe and known-unsafe certification, trace schema, the activity proof against brute force, boundary localization, and a reproducible end-to-end attack.

5 Does the complete verifier inherit the gap?

We also ran the constructed instance through α,β-CROWN's full solver, `ABCrownSolver.verify()`, on CPU with the specification $y_0 > y_1$ over the box. Its attack stage found the corner (every coordinate −0.05) with margin −8 and returned `falsified` (`unsafe-pgd`) in about 8 ms, before any bound computation or branch-and-bound ran. The complete pipeline therefore did not issue a false SAFE, so our claim is scoped to a float32 discrepancy in CROWN's bound computation, which the full verifier's attack stage caught. Whether its bounding path alone would certify this instance, with the attack stage disabled, is the natural next experiment.

Limitations. The violating network is constructed, not trained; a small MNIST-trained network would make the negative control more representative. The activity proof is exact for one hidden layer; deeper networks would need interval propagation, which gives a bound rather than an exact minimum.

References

[1] K. Jia and M. C. Rinard. Exploiting Verified Neural Networks via Floating Point Numerical Error. *Static Analysis Symposium* (SAS), 2021. arXiv:2003.03021.

[2] H. Zhang, T.-W. Weng, P.-Y. Chen, C.-J. Hsieh, and L. Daniel. Efficient Neural Network Robustness Certification with General Activation Functions. *NeurIPS*, 2018.

[3] K. Xu, Z. Shi, H. Zhang, Y. Wang, K.-W. Chang, M. Huang, B. Kailkhura, X. Lin, and C.-J. Hsieh. Automatic Perturbation Analysis for Scalable Certified Robustness and Beyond. *NeurIPS*, 2020.

[4] K. Xu, H. Zhang, S. Wang, Y. Wang, S. Jana, X. Lin, and C.-J. Hsieh. Fast and Complete: Enabling Complete Neural Network Verification with Rapid and Massively Parallel Incomplete Verifiers. *ICLR*, 2021.

[5] S. Wang, H. Zhang, K. Xu, X. Lin, S. Jana, C.-J. Hsieh, and J. Z. Kolter. Beta-CROWN: Efficient Bound Propagation with Per-Neuron Split Constraints for Neural Network Robustness Verification. *NeurIPS*, 2021.

[6] J. Jin, O. Ohrimenko, and B. I. P. Rubinstein. Getting a-Round Guarantees: Floating-Point Attacks on Certified Robustness. arXiv:2205.10159, 2022.

[7] A. Szász, B. Bánhelyi, and M. Jelasity. No Soundness in the Real World: On the Challenges of the Verification of Deployed Neural Networks. *ICML*, 2025.

[8] α,β-CROWN Team. alpha-beta-CROWN, release 0.8.0 (commit b58d83c). https://github.com/Verified-Intelligence/alpha-beta-CROWN